

Part A — Read a task table

Task	Function	Period (ms)	WCET (ms)	Deadline (ms)
T1	sensor read	10	1.0	10
T2	control loop	20	3.0	20
T3	comms send	50	5.0	50
T4	logging	100	10.0	100

CPU fraction:

Utilization = [WCET] / [Period]

T1: 1ms/10ms = 0.10 or 10%

T2: 3ms/20ms = 0.15 or 15%

T3: 5ms/50ms = 0.10 or 10%

T4: 10ms/100ms = 0.10 or 10%

Sum = 10% + 15% + 10% + 10% = 45% or 0.45

Idle fraction leftover= 100% - 45% = 55% or 0.55

Priority ranking:

(Highest) T1 → T2 → T3 → T4 (Lowest)

The rule I used was that tasks with shorter periods receive higher priority than tasks with longer periods because they would have more frequency.

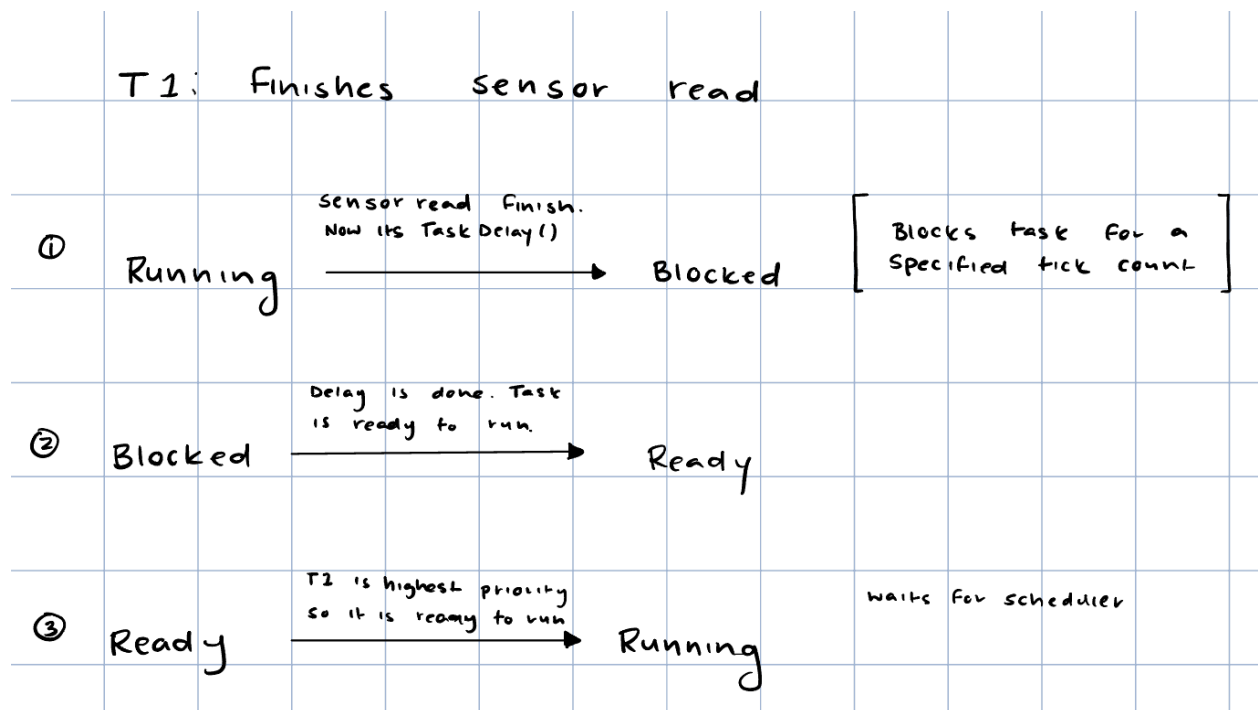
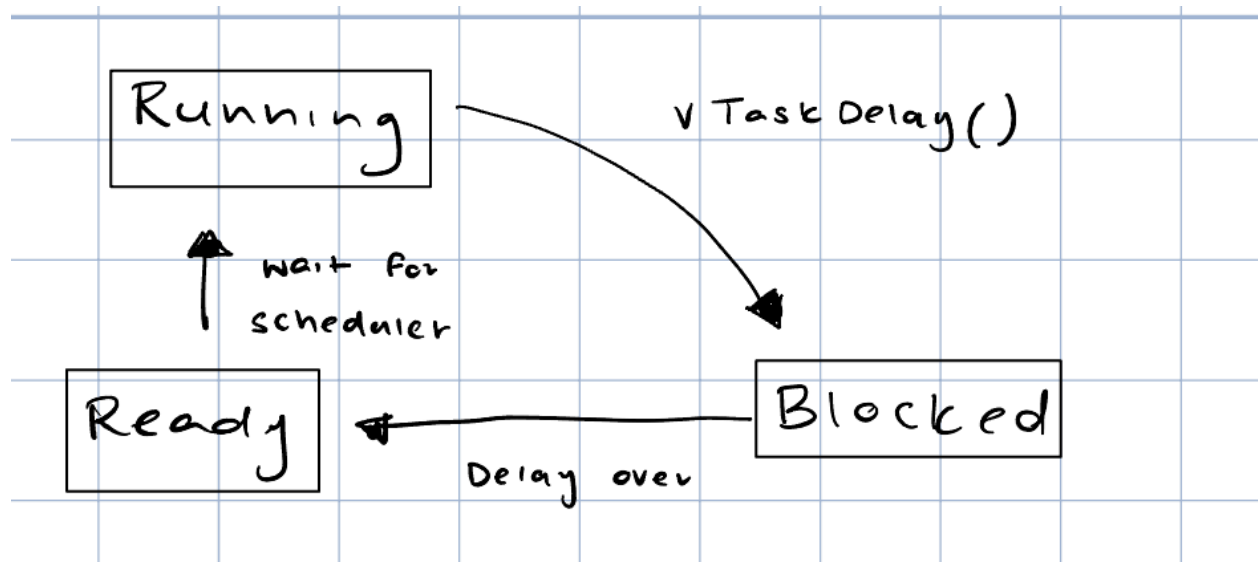
Task	Priority Bands	Justification
T1	10-14 ~ Sensor/IPC	It says sensor as the function but it also has the shortest period indication that it probably handles data. It would have a high priority.
T2	15-18 ~ Control	It also says control in the function name. This control loop probably will handle a lot of core logic and algorithms. While the priority band is higher than T1 (Sensor/IPC), T1 would still hold the highest priority as it handles sensor data and still has the shortest period.

T3	5-9 ~ User	I am going to assume the “comms send” does not refer to sending sensor data because the period is so long, it would be shorter if it were transmitting live raw sensor data. Due to this, it would probably be default user tasks handling normal network and communication transmission.
T4	1-4 ~ Housekeeping	I am going to use the same reasoning as Task 3. Because the period is so long the “logging” should not be anything important; therefore, it would probably be logging miscellaneous data in the background making it housekeeping.

Yield semantics:

Its loop should yield with vTaskDelayUntil because it eliminates drift by using a fixed tick count to stop lag, and also corrects any phase error by ensuring tasks are synced after preempted tasks.

State-machine trace:



Part B — xTaskCreate defened

```

xTaskCreate(
    vControlLoop,    // Function
    "ControlLoop",  // Task name
  
```

```

//[2048 bytes] / [4 bytes per word ] = 512 words
512,    //Stack size (WORDS)

NULL, //Pointer Parameters

16,    //Priority

NULL,   //Task Handler

);

```

Defend Parameters:

vControlLoop : arbitrary name of loop function defined in the table

“ControlLoop” : name of task

512 : I estimated the worst-case stack depth as 512. As stated in the slides, the max stack is 8KB, i just quartered that to have 2048 bytes (2KB stack) with 4 bytes per word which is 512 words

NULL : there's no pointers or variables passed to this task

16 : is the priority number, the priority band is 15-18, so I chose the lower median value

NULL : there is no handler or anything else so its null

Part C — Design your own

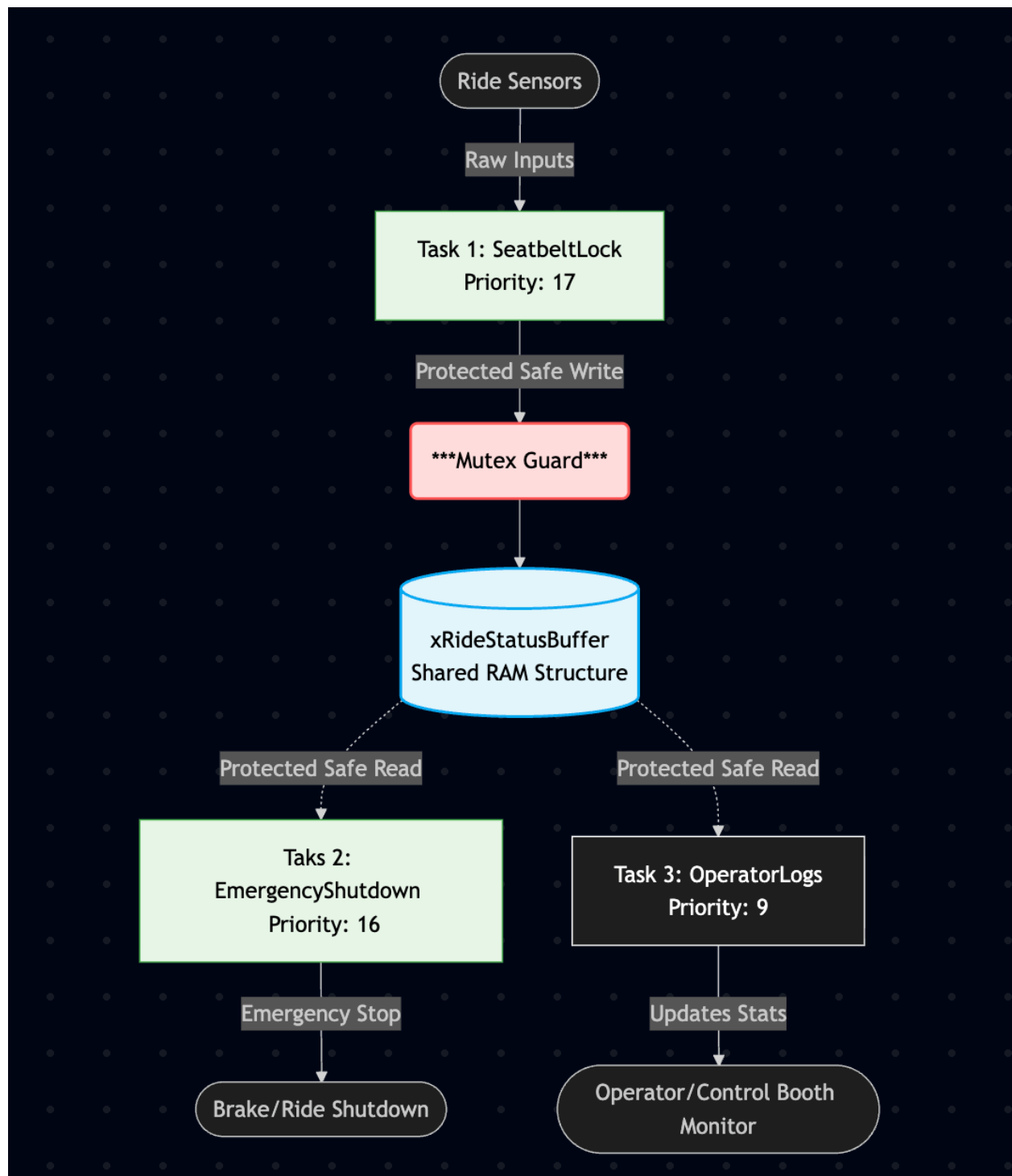
Theme : Industrial/Theme-park

Name	Purpose	Period	WCET	Deadline	Priority	Justification
T1 - SeatbeltLock	Detects if the rider is in the cart and that the harness or lap bar is locked before the ride starts	20ms	1ms	20ms	17	Very hard deadline that requires highest priority because it results in rider safety

T2 - Emergency Shutdown	Monitors the tracks and cart positions, initiating emergency breaks if something goes wrong.	20ms	2ms	20ms	16	Same priority as the seatbelt lock task because it's the same hard deadline that is responsible for public safety.
T3 - OperatorL ogs	Updates the operator screen with ride telemetry and statuses.	100ms	10ms	100ms	9	Lower priority, it updates the operator screen which is a firm/soft deadline. It does not cause a system shutdown but the operator must wait for the screen to update to control the ride.

One shared resource between tasks could be rider telemetry data or status that shows live sensor data from the seatbelt locks which can be shared with the SeatbeltLock, EmergencyShutdown, and OperatorLogs tasks. This shared resource can be used to update the 3 tasks.

Guarding: FreeRTOS Mutex - since both tasks are able to access this resource, there needs to be mutex to make sure the data does not get corrupted.



Part D — Industry anchor

AUTOSAR Classic:

AUTOSAR Classic uses a fixed-priority preemptive scheduling model to run its tasks. It's used in the automotive industry, running the Electronic Control Units (ECUs); specifically designed for safety systems with hard real-time requirements that manage things like your engine, steering, and anti-lock brakes when you drive. We use this technology everyday without even realizing it, which is why I chose to write about it

A difference from FreeRTOS is how memory and tasks are allocated. FreeRTOS allows you to dynamically create and destroy tasks and queues at runtime while the system is running. AUTOSAR Classic forbids runtime allocation for safety reasons, meaning every task, stack size, and memory boundary must be hardcoded and remain so at compile time before the car is ready to sell or drive just as listed in the JPL power of 10 rules

Sources used:

<https://www.autosar.org/standards/classic-platform>

<https://www.ul.com/sis/blog/autosar-part-1-classic-platform>